

## Article

# Small Language Model-Guided Quantile Temporal Difference Learning for Improved IoT Application Placement in Fog Computing

Bhargavi Krishnamurthy <sup>1,\*</sup>  and Sajjan G. Shiva <sup>2,\*</sup> <sup>1</sup> Department of Computer Science and Engineering, Siddaganga Institute of Technology, Tumakuru 572103, India<sup>2</sup> Department of Computer Science, University of Memphis, Memphis, TN 38107, USA

\* Correspondence: bhargavik@sit.ac.in (B.K.); sshiva@memphis.edu (S.G.S.)

## Abstract

The global market for fog computing is expected to reach USD 6385 million by 2032. Modern enterprises rely on fog computing since it offers computational resources at edge devices through decentralized computation mechanisms. One of the crucial components of fog computing is the proper placement of applications on fog nodes (edge devices, Internet of Things (IoT)) for servicing. Large-scale, geographically distributed fog networks and heterogeneity of fog nodes make application placement a challenging task. Quantile Temporal Difference Learning (QTDL) is a promising distributed form of a reinforcement learning algorithm. It is superior compared to traditional reinforcement learning as it learns the act of prediction based on the full distribution of returns. QTDL is enriched by a small language model (SLM), which results in low inference latency, reduced costs of operation, and also enhanced rates of learning. The SLM, being a lightweight model, has policy-shaping capability, which makes it an ideal choice for the resource-constrained environment of edge devices. The data-driven quantiles of temporal difference learning are blended with the informed heuristics of the SLM to prevent quantile loss and over- or underestimation of the policies. In this paper, a novel SLM-guided QTDL framework is proposed to perform task scheduling among fog nodes. The proposed framework is implemented using the iFogSim simulator by considering both certain and uncertain fog computing environments. Further, the results obtained are validated using expected value analysis. The performance of the proposed framework is found to be satisfactory with respect of the following performance metrics: energy consumption, makespan time violations, budget violations, and load imbalance ratio.

**Keywords:** fog computing; application placement; quantile temporal difference learning; small language model

**MSC:** 35K34



Academic Editors: Marjan Mernik, Shiqiang Liu and Jonathan Blackledge

Received: 17 May 2025

Revised: 23 July 2025

Accepted: 21 August 2025

Published: 28 August 2025

**Citation:** Krishnamurthy, B.; Shiva, S.G. Small Language Model-Guided Quantile Temporal Difference Learning for Improved IoT Application Placement in Fog Computing. *Mathematics* **2025**, *13*, 2768. <https://doi.org/10.3390/math13172768>

**Copyright:** © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

## 1. Introduction

Fog computing is a distributed computing paradigm that offers processing at the edge devices as far as possible instead of depending completely on the cloud servers. It enhances the capability of cloud computing with faster decision-making capability, making it suitable for bandwidth-constrained applications. It overcomes the limitations of a cloud computing environment with real-time data processing, reduced latency, and local decision-making

capability. It allows the system to operate independently of the cloud, even in the case of connectivity failures. It offers scalable solutions by evenly distributing the load across the edge devices. Mobile fog devices also provide context-aware services that follow the end users wherever they move [1,2]. Because of these characteristics, fog computing has been adopted in a variety of applications, including smart cities, industrial IoT, retail, agriculture, transportation, etc.

Two key components of fog computing are fog nodes and fog network. A fog node is composed of computing capability, storage, and network connectivity. The fog network is responsible for establishing the communication infrastructure among fog nodes. The fog nodes perform data gathering, data analytics, and a short duration of storage and transmission of only a filtered form of data to cloud servers. They bring the computation near the origin of data to offer benefits in terms of reduced latency, enhanced security, and enriched efficiency. Local computation helps in the immediate processing of the input data to arrive at independent decisions at a faster pace. The servicing delay and computation cost are minimized as they offer high scalability by utilizing the resources available in edge devices. The overall responsiveness is high as they reduce the bandwidth requirement by reducing the data transmission rate to remote cloud servers. Mobility among the fog nodes makes it suitable for Internet of Things (IoT) applications. Mobility is ensured through application communication with stationary fog nodes as well as vehicle-to-vehicle communication. The network topology is highly dynamic, and fog nodes can enter and leave the network easily [3,4].

Heterogeneity has become the fundamental principle of fog computing. The fog nodes exhibit varying resource capacity, bandwidth availability, and power constraints. The geographically distributed form of fog nodes offers redundancy and depends on the cloud connectivity only for coordination purposes. The fog nodes are often subjected to failures, poor connectivity, improper placement of applications, and disruption in the network services. Failures among the fog nodes are quite common because of wide geographical distribution, a distributed form of deployment, battery-constrained computing, and overflow of resources.

Since the edge computing paradigm is distributed in nature, the optimal placement of applications across fog nodes is a challenging task as the application placement policies must operate under the constraints of energy consumption, computation overhead, and bandwidth usage. The application placement typically involves two steps, i.e., mapping and automatically adjusting. In the mapping step, the application is mapped to a specific fog node, and sufficient resources are provided for execution. During the automatic adjustment step, horizontal scaling or vertical scaling is performed to adjust to the varying resource demands. An application is composed of activities such as data receiving, preprocessing, data analysis, and event handling. Most of the modern applications are non-monolithic in nature as they are composed of several independent deployable components. In order to make application placement more adaptable, it is necessary to consider application scaling and proper task distribution to deal with the changing volume of incoming requests [1,5].

The traditional application placement policies strive to meet the quality of service requirements in terms of latency, bandwidth, energy consumption, and service level agreement (SLA) violations. However, the application requirements keep varying, and it is difficult to predict them. Fog applications usually exhibit stringent quality of service (QoS) requirements. In addition to QoS requirements, security, privacy, location sensitivity, deadline sensitivity, state persistence, and fault tolerance requirements also need to be taken into consideration. Three different modes are considered for application placement: batch scheduling, static scheduling, and dynamic scheduling. In batch scheduling, application requests are batched and then transferred for execution. In static scheduling, applications are

scheduled without considering the current system operation status. Dynamic scheduling considers both static system counterparts and short term variations to arrive at placement decisions. The entire application placement strategy is handled by the controller node which is situated either in the fog layer or in the cloud layer. Multiple controller nodes may be used to coordinate and arrive at placement decisions. However, finding the best fog node for application placement is difficult as there is no universal definition for it and it varies depending on the application needs [6,7].

The reinforcement learning technique is increasingly integrated with the edge computing paradigm as it significantly improves the intelligence factor, flexibility, and adaptability nature of edge systems. Incorporating machine learning at the edge devices enables immediate data analysis and reduces the overall latency incurred. Local decisions are taken up at edge nodes which makes them reliable and minimizes the risk of data breaches. Quantile Temporal Difference Learning (QTDL) is a distributed form of reinforcement learning technique which learns the art of prediction from full distribution of returns instead of only the expected value return. It is not possible to learn return distributions in an infinite dimensional space. Hence, QTDL learns approximation of return distribution by choosing probability distribution representation and uses a subset of parameterized finite dimensional set of parameters. It aims to learn certain quantiles from of return distribution from every state of the Markov Decision Process (MDP). QTDL is highly nonlinear in nature and is made up of multiple fixed points. However, it often ends up in suboptimal policies due to overestimation of value associated with certain actions. Computational complexity is high due to frequent update of quantile values. Accurate estimation of quantile values is difficult since a large number of samples representing the state of fog nodes and fog network are needed. Temporal difference error (i.e., wrong estimation of value function and reward gathered from transition) may also occur [8,9]. The Small Language Model (SLM) is a scaled-down version of the Large Language Model (LLM). It is a lightweight model with fewer parameters to tune which offers a viable option in terms of enhanced efficiency and reduced cost of computation. The working principle of SLM is the same as that of LLM which includes several layers of attention mechanisms and also reduces the size of the dataset through mechanisms like knowledge distillation, pruning, quantization, and deployment of expert-regulated architectures. The computational efficiency of SLM is higher than that of LLM as it involves fewer parameters and offers lower memory requirements [10,11]. The SLM-guided QTDL effectively handles the ambiguous and noisy state space environment. It enhances the uncertainty modeling through exploration over the environment with high variance and sparse rewards. Heuristic guidance is required to handle multimodal inputs and to prevent early the tendency to take meaningless or misleading actions. The data-driven quantiles of temporal difference learning are blended with the informed heuristics of the SLM to prevent quantile loss and over- or underestimation of the quantile ranks.

In this paper, an SLM-guided temporal difference learning framework is proposed to perform optimal application placement in fog computing. The QTD learning performs approximation by assigning an equal weighted mixture of Dirac deltas. The particle locations are approximate as are also the certain count of quantiles. The heuristic value generated by the SLM is used to learn an exact Q value of the TD learning. It prevents hallucination by avoiding improper guidance to the TD learning agent. The heuristic-guided Q function is converged with optimal Q function of MDP. Also SLM dynamically adapts to the change in the application requirements with few computational resources through efficient exploration of the state space environment. The SLM-generated Q value offers a greater level of flexibility through proper reshaping of reward function. The bias in

the TD Q learning agent's performance is prevented by providing adequate guidance to arrive at the optimal solution.

The main objectives of this paper are as follows.

- Design and development of the lightweight model, i.e., SLM-enabled QTDL framework for efficient placement of fog node applications.
- Utilization of the heuristic value generated by SLM to aid QTDL in arriving at exact Q value computation and learning.
- Prevention of the tendency of TD learning agent to become trapped in hallucination through proper guidance and fine tuning of the parameters.
- Proper reshaping of QTDL reward function through precise estimation of Q function of QTDL agents.
- Enhancing the flexibility and adaptability of QTDL framework by achieving the trade-off between exploration and exploitation of the large state space of the fog environment.
- Expected value analysis of the QTDL framework considering finite and infinite fog computing environment.

Experimental evaluation of the QTDL framework using iFogSim simulator, (developed by Cloud Computing and Distributed Systems (CLOUDS) laboratory at the University of Melbourne, Australia). The performance metrics considered for evaluation are energy consumption, makespan time violation, budget violation, and load imbalance.

The remaining parts of this paper are organized as follows. Section 2 discusses related work; Section 3 presents the proposed work; and Section 4 deals with results and discussion. Section 5 gives expected value analysis, and finally, Section 6 provides the conclusions.

## 2. Related Work

In this section, brief descriptions of recent papers on application placement among nodes in fog computing are provided and their limitations are identified.

Mohammed et al. presented a memetic algorithm-based model for placement of applications in an edge computing environment [12]. Each application is partitioned into different forms of granularity which include class and task. The application placement in fog computing is considered as a non-polynomial time-hard problem. The conventional memetic algorithm is modified to provide a reasonable solution for the application placement problem within a predefined decision time. The memetic algorithm is an extension of the conventional evolutionary algorithm and mainly aims to accelerate evolutionary search space to arrive at the optimum solution. Memetic algorithm pairs up with the conventional genetic algorithm with two refinement methodologies: local search and individual learning. Each is responsible for representing a candidate solution and later the potential solution is extracted from the population of candidate solutions. Five main steps are involved in the memetic algorithm: initialization, selection, crossover, mutation, and local search procedure. The application placement is composed of three strategies which include pre-scheduling, batch placement of application, and recovery from failure. During the pre-scheduling phase, the broker unit arranges the concurrent applications for execution. The batch application placement of application is carried out using an optimized version of the memetic algorithm. Potential failures during runtime are overcome using the lightweight failure recovery methodology. The overall execution time and energy consumption of the applications are minimized using a weighted cost model. However, the approach is prone to premature convergence and higher computational cost as it employs a local instead of global search.

Zoltan et al. discussed a decentralized approach for application placement in fog computing [6]. Factors causing difficulty in application placement are the huge number

of application and infrastructure nodes. The main aim of decentralization is to operate separately over different parts of the architecture to enhance scalability. Fog colonies are constructed composing resources that are spread across the geographical region. The number of fog nodes in each colony is dynamically distributed and overlapping of fog nodes is permitted. Applications are assigned to the non-overlapped fog nodes to retain a higher level of flexibility. Good cooperation among the fog colonies is established which prevents missed synergy and also avoids convergence towards suboptimal solutions. An integer linear programming algorithm is utilized as a feasibility approach that restricts all variables to be integers. At first, a problem instance is selected and converted into an integer linear program. Then, an external solver is applied to arrive at a new application placement and migration policies. The host is identified for all newly added applications, and the hosts are arranged in an ascending order. First, the application is placed on to the first host and if it is not feasible, migration decisions are made using integer-valued logic. The dynamically distributed fog colonies are paired with good coordination among them which leads to promising results in terms of resource utilization. However, uncertainty and nonlinearity are not taken into consideration, which makes it impossible to adapt to higher dimensionality problems.

Heba et al. discussed a Quality of Experience (QoE)-based prioritization model for application placement in fog computing [13]. Higher round-trip time in data processing usually affects user perceptions of the Quality of Service (QoS). User context and application runtime context are taken into consideration. Environment runtime context is influenced by parameters like location, network strength, and battery level. Similarly, the application runtime context is influenced by parameters like usefulness, ease of use, boring, enjoyable, and monotonous. The user expectation is judged based on reliability, sustainability, and availability. At first, priority is assigned for applications considering runtime, application usage, and user expectations for quality. Then, the applications are allocated to fog nodes considering the proximity, computation capability, and intended response time. QoS violation causes a direct influence on the application placement strategy. Recalculation and reprioritization of the applications happen, and the updated plan will be conveyed to the fog action implementer. The quality rating gain metric plays a major role in prioritizing the applications. It is determined by the product of QoE and the score of the computing capability. The watchdog is responsible for receiving the update information and checking for Service Level Violations (SLVs) and critical changes in the context of operation. This approach improves the QoE of the users with respect to application placement time and the power consumption is reduced. However, the approach is not suitable for real-time fog environments since there is a chance of a higher priority application starving for an infinite period of time due to the influence of uncertainty among fog nodes. Also, the power consumption of fog nodes is higher in the urge to meet the service delivery deadline. Compatibility of the solution across multiple applications is also less since different categories of applications are not taken into consideration.

Reinout et al. presented a multiple objective-based application placement approach which aimed to gain a maximum reward formulation [14]. The application placement problem is considered as a meta learning problem where task distribution is performed over distribution of preferences. The application placement approach is trained considering the meta policies. The meta policy is trained simultaneously by considering a huge collection of task distributions which are randomly sampled over Markov Decision Process (MDP). A trained neural network model is deployed to reduce the resource consumption rate of the application placement logic. The few shot adaptation to the new application is supported by giving relative preference to multiple objectives. The generic version of the Bellman equation is utilized to formulate a single parametric representation of optimal policy over



the large state space of preferences. During the initial stage of learning, the Q learning agent performs optimal policy for any given preference under few samples of preferences. The series of loss functions are minimized using homotopy optimization. The efficient alignment between the application preferences and optimal solution is derived by reusing the transitions from the learning stage using hindsight experience replay. Since computing of optimality filter over the Q table is not possible, mini batch transition is performed before updating the parameters. The main goal is to draw parameters from Gaussian distribution to perform policy which is a combination of stochastic search and policy gradient. The results are formulated using pareto solutions which ensure solutions are correct in terms of optimality and computational efficiency. The performance is found to be good over continuous control problems which are enriched with high degree of freedom. However, the approach lacks scalability as convex approximation is performed over optimal policies to efficiently handle individual policies which grow linearly along with the increase in domain size.

Mohammed et al. presented a distributed deep learning approach for application placement in fog computing [15]. The incoming applications are modeled as directed acyclic graphs composed of diverse topologies. The application placement complexity is very high as dependency among applications involves several additional constraints pertaining to resource requirement. Here, an actor critic-distributed Q learning algorithm is designed which is dependent on weighted actor–learner architectures. Two types of neural networks are used to optimize the learning process. Actor neural network selects a random action which is based on the current policy. Similarly, the critic neural network estimates a value (return) for those actions. It is a form of deterministic implementation which completes its segment of experience before performing average operation over all the actors' actions. The agent is capable of communicating value and policy parameters to other agents which diffuse the information across the deep neural network. Robust exploration of the state space is carried out by achieving a proper tradeoff between exploration and exploitation activities. The application features are extracted and represented in the form of MDP. The rewards are formulated in such a way that they directly determine the effectiveness of the actions and reduce the application completion time. The exploration cost of the Q learning agent is reduced using distributed experience trajectory generation. An off-policy correction methodology is employed to converge to an optimal solution at a faster rate. The approach is highly scalable as the computational cost depends on neighboring agents instead of all agents across the network. However, it is computationally very expensive as it requires both actor and critic networks to be trained simultaneously. Also, chances of instability and divergence are higher due to poor function approximation because of frequent interference between the actor agent and critic agent.

Xueying et al. presented a deep reinforcement learning-enriched federated learning algorithm to perform both task scheduling and resource allocation in an edge environment [16]. The internet of vehicles is a real-time demanding environment which consists of constraints with respect to time and varying environments. The main aim of developing an intelligent transport system is to integrate the sensing and communication mechanisms in order to enhance the rate of communication between the mobbing vehicles. A federated learning algorithm which is self-supervised in nature is capable of offloading partially completed tasks onto the roadside unit to reduce the energy consumption rate among the internet of vehicles. The incoming tasks are divided into local and roadside tasks. The training of the roadside units and local computation units is balanced. A threshold is set for offloading of tasks which prevents the inefficiencies in terms of over- or under-offloading of tasks. Smooth adjustments are made between transmission power and task assignment ratio to minimize the energy consumption resulting in enhanced efficiency. Proper trade-

off between exploration and exploitation phases is achieved through maximum entropy mechanism. The deep Q network is used to reduce the overestimation of bias during value function approximation. Failure during the training phase and premature convergence to a local optimal solution is prevented as the federated learning agent continuously attempts to formulate new policies. However, the data model includes security vulnerabilities like model poisoning, leakage of private data, and higher system level risks. It also exhibits potential bias during decision making because of higher communication costs and poor representation of local data.

To summarize, the existing works exhibit the following drawbacks:

- Most of the approaches fall into premature convergence and higher computational cost due to improper distribution of applications among fog nodes.
- Inability to handle uncertainty among the application requirements and fog node processing capability due to frequent violation of SLAs.
- Higher tendency to produce sub-optimal solutions due to poor exploration of the state space of fog environment.
- Higher loss function ratio due to single parametric representation of the application and resources at fog nodes.
- Lesser compatibility of application placement policies as the diverse requirement of the applications are not taken into consideration.
- The decisions taken up by the traditional machine learning algorithms are subjected to bias due to reinforcement of assumptions which does not align with the objective functions.

### 3. System Model

The fog computing environment is a decentralized computing environment composed of data, storage, computation, and applications that are placed in between data source and cloud server. Billions of IoT objects are used in everyday life which are deployed extensively and exhibit a high level of connectivity and computation intelligence [17]. One of the primary requirements of fog computing is the optimization of the application placement to ensure smooth functioning.

Consider an  $m$  set of applications, i.e.,  $IoT_{App} = \{IoT_{App_1}, IoT_{App_2}, IoT_{App_3}, \dots, IoT_{App_m}\}$ , and an  $n$  set of fog nodes, i.e.,  $FN = \{fn_1, fn_2, \dots, fn_n\}$ . Each of the applications is represented considering attributes like deadline  $DL(App_i)$ , processing power  $PP(App_i)$ , storage capability  $SC(App_i)$ , memory capacity  $MC(App_i)$ , and network traffic usage  $NTU(App_i)$ , i.e.,  $IoT_{App_i} = \langle DL(App_i), PP(App_i), SC(App_i), MC(App_i), NTU(App_i) \rangle$ . Similarly, each of the fog nodes is represented considering the resource-related attributes like CPU capability  $CPU(fn_i)$ , RAM capacity,  $RAM(fn_i)$ , disk storage  $DS(fn_i)$ , and network bandwidth,  $NB(fn_i) \rangle$ , i.e.,  $fn_i = \langle CPU(fn_i), RAM(fn_i), DS(fn_i), NB(fn_i) \rangle$ . The SLM-guided QTDL framework Heuristic Q buffer:  $D(G(P)) = \{(s_i, a_i, Q_i) | (s_i, a_i, Q_i) \in D(G(P)), i = 1, 2, 3, \dots, n\}$  is initialized. The quantile regression loss  $E_z(\tau | \{Z \geq v\} + (1 - \tau) | \{Z < v\}) | Z - v |$  is determined for each distribution  $v$ , where  $v$  is the target value,  $Z$  is the predicted quantile value, and  $\tau$  is the quantile level which ranges between 0 and 1. Compute quantile regression loss  $L_x^{\tau, \pi}(v) = E_x^{\tau}(\tau | \{\Delta \geq 0\} + (1 - \tau) | \{\Delta < 0\}) | \Delta |$ , where  $\sum_{t=0}^{t-\infty} \gamma^t R_t - v$ . Update the rule  $\theta(x, i) = \theta(x, i) + \alpha(\tau_i | \{\sum_{t=0}^{t-\infty} \gamma^t R_t \geq \theta(x, i)\} - (1 - \tau_i) | \{\sum_{t=0}^{t-\infty} \gamma^t R_t \leq \theta(x, i)\})$ , where  $\theta(x, i)$  is the  $i$ th predicted quantile value of state  $x$ ,  $\alpha$  is the learning rate. The application placement policies, i.e.,  $IoT\_Pol = \{IoT_{App_1} \rightarrow fn_1, IoT_{App_2} \rightarrow fn_2, IoT_{App_3} \rightarrow fn_3, \dots, IoT_{App_m} \rightarrow fn_n\}$  are formulated.

The Performance Metrics (PMs) considered for the evaluation of the proposed SLM\_QTDL framework are defined as follows:

PM1: Total Energy Consumption (TEC (SLM\_QTDL)): The total energy consumption of the SLM\_QTDL is the summation of the static energy consumption and dynamic energy consumption at time interval  $\Delta T$  [17].

$$\text{TEC}(\text{SLM\_QTDL}) = \text{SEC}(\text{SLM\_QTDL}, \Delta T) + \text{DEC}(\text{SLM\_QTDL}, \Delta T) \quad (1)$$

where the static energy consumption represents the energy consumed by the fog nodes when it is active.

$$\text{SEC}(\text{SLM\_QTDL}, \Delta T) = \begin{cases} \alpha, & fn_i = \text{Active}, \text{ where } 1 \leq i \leq n \\ 0, & fn_i = \text{Passive}, \text{ where } 1 \leq i \leq n \end{cases}$$

Similarly, the dynamic energy consumption represents the energy consumed by the fog nodes while servicing the applications. It is computed by the product of CPU capability of the fog node during the time interval  $\Delta T$   $fn_i^{CPU}(\Delta T)$  and maximum energy consumption of the fog node  $\text{MAX}(\text{EC}(fn_i))$  and

$$\text{DEC}(\text{SLM\_QTDL}, \Delta T) = \sum_{fn_i \in FN} fn_i^{CPU}(\Delta T) * \text{MAX}(\text{EC}(fn_i))$$

PM2: Makespan Time Violation (MSV(SLM\_QTDL)): Makespan time is the total processing delay and communication delay incurred while servicing an application. It is measured through the difference between maximum value of makespan time incurred to process the application using the SLM\_QTDL  $\text{MAX}(\text{MS\_SLM\_QTDL})$  and makespan time as defined by the QoS profile of the application  $\text{MS\_QoS}(App_i)$ .

$$\text{MSV}(\text{SLM\_QTDL}) = \sum_{\forall App_i \in APP} \text{MAX}(\text{MS\_SLM\_QTDL}) - \text{MS\_QoS}(App_i) \quad (2)$$

where

$\text{MS\_SLM\_QTDL} = \text{MS}_{nw}(\text{SLM\_QTDL}) + \text{MS}_{proc}(\text{SLM\_QTDL})$ , in which the  $\text{MS}_{nw}(\text{SLM\_QTDL})$  is the network latency and  $\text{MS}_{proc}(\text{SLM\_QTDL})$  is the processing latency.

PM3: Budget Violation (BV(SLM\_QTDL)): Budget is calculated by considering the cost involved in servicing the applications. Budget violation is measured through the product of cost of executing the application on fog node  $\text{Cost}_{App_i}^{fn_i}$  and pricing model followed at the fog node  $\text{Price}_{fn_i}$ .

$$\text{BV}(\text{SLM\_QTDL}) = \sum_{\substack{\forall App_i \in APP \\ fn_i \in FN}} \text{Cost}_{App_i}^{fn_i} * \text{Price}_{fn_i} \quad (3)$$

PM4: Load Imbalance (LI(SLM\_QTDL)): The load over the fog node is computed as the weighted sum of the different type of resources utilized by the fog nodes, i.e., CPU utilization  $fn_i^{CPU}(\Delta T)$ , memory utilization  $fn_i^{MU}(\Delta T)$ , and bandwidth utilization  $fn_i^{BU}(\Delta T)$ .

$$\text{Load}(\text{SLM\_QTDL}) = \left( \alpha_1 * fn_i^{CPU}(\Delta T) + \alpha_2 * fn_i^{MU}(\Delta T) + \alpha_3 * fn_i^{BU}(\Delta T) \right) > \text{CP}(FN) \quad (4)$$

where  $\alpha_1$ ,  $\alpha_2$ , and  $\alpha_3$  are the weights assigned to different type of resources,  $\alpha_1 + \alpha_2 + \alpha_3 = 1$ .

$$\alpha_1 = \frac{fn_i^{CPU}(\Delta T)}{fn_i^{CPU}(\Delta T) + fn_i^{MU}(\Delta T) + fn_i^{BU}(\Delta T)}$$

$$\alpha_2 = \frac{fn_i^{MU}(\Delta T)}{fn_i^{CPU}(\Delta T) + fn_i^{MU}(\Delta T) + fn_i^{BU}(\Delta T)}$$



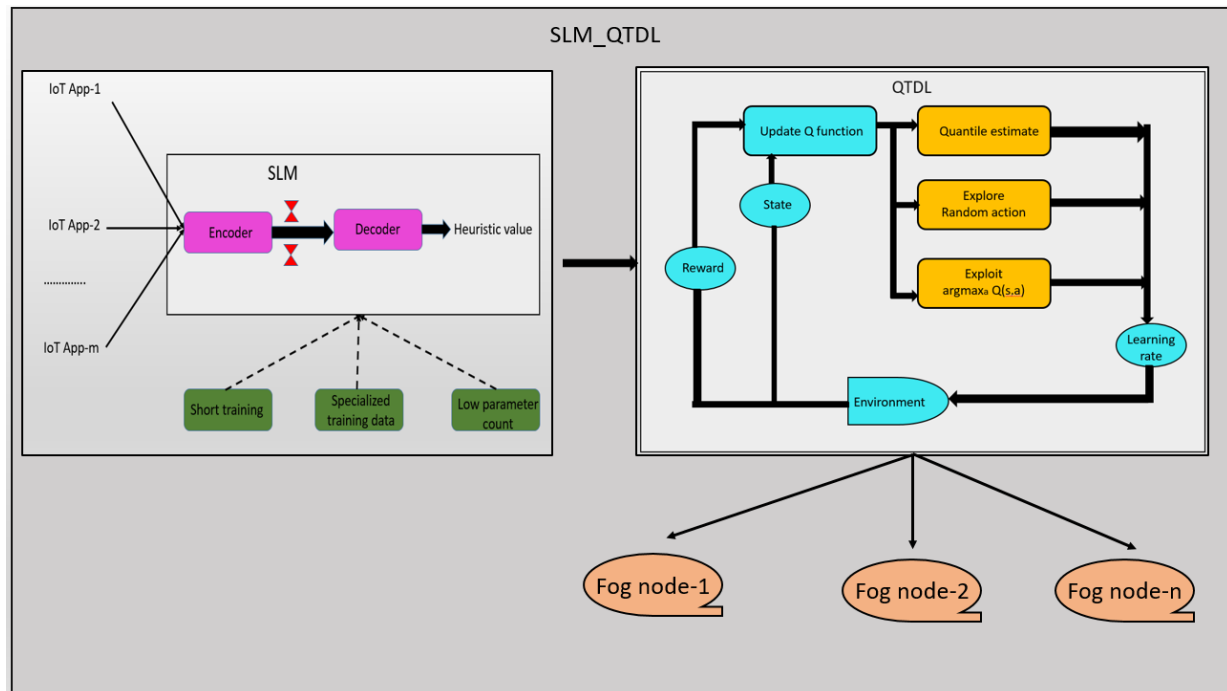
$$\alpha_3 = \frac{f n_i^{BU}(\Delta T)}{f n_i^{CPU}(\Delta T) + f n_i^{MU}(\Delta T) + f n_i^{BU}(\Delta T)}$$

#### 4. Proposed Work

The architecture of SLM-guided QTDL framework for application placement in fog computing is shown in Figure 1. The framework is composed of two modules: SLM and QTDL. The SLM module inputs the applications with varying resource requirements. The SLM-enriched QTDL learning agent is capable of performing specific tasks over the edge devices which have fewer resources. The foundational elements of QTDL are state (s), action (a), and reward (r). State represents the environment status of the temporal difference learning agent for a given time  $t$ . The state includes metrics like bandwidth, CPU capacity, task queue length, and memory. Action indicates the decision operation performed in response to the current system state. The actions might include changing the task scheduling policies, allocating fewer or more resources, and offloading the tasks to appropriate fog nodes. Reward is the feedback signal obtained due to the action performed by the agent. It basically provides guidance of the agent to exhibit a balanced energy efficient behavior. The balanced behavior of the agent is measured by considering improper penalties, energy consumption, and task completion time. An array of SLMs are capable of exhibiting the functionality of an LLM but they are tiny in size and demand smaller dataset for training. The quantile learning over the distribution of returns is more accurate as it can precisely estimate the tradeoff between variance updates and asymptotic predictions. After performing a sufficient amount of estimation over the quantiles, the agent determines the true value function. The integration of SLM with QTDL is a powerful combination of compact language-guided reasoning mechanism with the distributional value estimation capability. This integration is useful in developing frameworks for resource-constrained environments like fog computing. The design principles followed to develop the framework are as follows. The SLM is used to parse and understand the application requirements. Low latency and lesser memory requirements are ensured through contextual embeddings which provide proper guidance for action selection and value estimation. Latent representation alignment is carried out to ensure that the SLM output embeddings are compatible with the quantile network input space. The number of quantiles used are limited by tiny, distilled models. Quantized SLM models perform on-device inference and use the recent quantile outputs for experience replay. Quantile outputs are used to express the uncertainty over the actions which are combined with the language-derived intent to provide explanation over the actions. SLM generates heuristic values which serve as a learned function to guide the QTDL agent based on the application's understanding. Basically, the SLM interprets the state description of the applications and outputs the scalar value which presents the desirability of the action. The state and action are encoded and passed through the SLM. The output layer of the SLM is fine-tuned using the feedback of reinforcement learning agent to output the heuristics score.

The detailed working of the framework is shown in Algorithm 1. The working of the framework is divided into training and testing phases. During the training phase, quantile regression loss and gradient loss are computed and updated accordingly. The Dirac delta approximation is used to compute the value of quantiles. Quantile regression loss is used to estimate the distribution of returns. Quantile values are updated using gradient loss estimators. Similarly during the testing phase the updated quantile rule is formulated through unbiased estimation of regression loss and gradient loss. The quantile loss will be re-evaluated, and update rules are formulated for each of the applications. The quantile estimate is refined to arrive at precise application placement policies. The QTDL agent can store the collection of predictions at each state of operation  $\theta(x, i)$ , which is the core

component of the agent's functionality. The agent keeps observing every ongoing transition  $(s, a, r, s')$  in the state space environment and performs update operations over all the Q value estimates whose learning rate is greater than or equal to zero. The temporal difference error is used for learning purpose which is analogous to the basic working principle of the traditional TDL. The influence of temporal difference error over the Q value updating process is determined through their sign, not on the magnitude. The resource allocation predictions made at each state are made to ensure the inclusion of distinct terminology. The inclusion of distinct terms and consideration of the sign makes the predictions accurate through properly learnt parameters. The framework scales easily with the increase in the size of the state space as each state of the agent is capable of storing multiple predictions. Even the small models lack visibility in terms of IoT devices' reliability, readiness, and resource availability. In-time monitoring of the computation resources in the distributed fog computing environment is required to ensure balanced distribution of applications, enabling the fault tolerant operations, and performing latency and energy aware application placement activity.



**Figure 1.** The high-level architecture of SLM-guided QTDL framework for application placement.

---

**Algorithm 1:** Working of SLM-guided QTDL framework for IoT application placement

---

1: **Start**

2: **Input:** A set of applications, i.e.,  $IoT_{App} = \{IoT_{App_1}, IoT_{App_2}, IoT_{App_3}, \dots, IoT_{App_m}\}$ , quantile estimate  $\theta \in R^{x*[m]}$ ,  $R$  is the reward set of dimension  $m$ , observed transition of state on reward  $(s, r, s')$ , i.e.,  $s$  is the current state,  $s'$  is the next state entered after receiving the reward  $r$ , Small Language Model (SLM), Prompt ( $P$ ), Heuristic Q buffer  $D(G(P)) = \emptyset$

3: **Output:** Application placement policies, i.e.,

$IoT\_Pol = \{IoT_{App_1} \rightarrow fn_1, IoT_{App_2} \rightarrow fn_2, IoT_{App_3} \rightarrow fn_3, \dots, IoT_{App_m} \rightarrow fn_n\}$ .

4: Generate Heuristic Q buffer:

$D(G(P)) = \{(s_i, a_i, Q_i) | (s_i, a_i, Q_i) \in D(G(P)), i = 1, 2, 3, \dots, n\}$

---

**Algorithm 1: Cont.****4: Training phase of SLM\_QTDL**

5: Set  $\tau_i = \frac{2i-1}{2m}$ , for each of the  $i$  ranges from 1 to  $m$

6: **For** each of the applications in the application set, i.e.,  $IoT_{App_i} \in IoT_{App}$  **do**

7:   Perform sampling over fog state space environment  $(s, a, r, s')$

8:   Update the generated Q buffer  $D(G(P)) = D(G(P)) \cup (s, a, r, s')$

9:   Sample the transition  $(s, a, r, s')$  from updated Q buffer  $D(G(P))$

10:   Compute approximation of equally weighted mixture of Dirac deltas for state  $x$ , i.e.,  $\delta_{\theta(x,i)}, \eta(x) = \sum_{i=1}^m \delta_{\theta(x,i)}$

11:   Determine quantile regression loss  $E_z(\tau|\{Z \geq v\} + (1-\tau)|\{Z < v\})|Z - v|$ , for each distribution  $v$ , where  $v$  is the target value,  $Z$  is the predicted quantile value,  $\tau$  is the quantile level which ranges between 0 and 1.

12:   Compute quantile regression loss  $L_x^{\tau, \pi}(v) = E_x^{\pi}(\tau|\{\Delta \geq 0\} + (1-\tau)|\{\Delta < 0\})|\Delta|$ , where  $\sum_{t=0}^{t-\infty} \gamma^t R_t - v$

13:   Unbiased estimator of negative gradient loss  $\tau|\{\sum_{t=0}^{t-\infty} \gamma^t R_t \geq v\} - (1-\tau)|\{\sum_{t=0}^{t-\infty} \gamma^t R_t \leq v\}$ , where  $v$  is the value threshold, penalizes overestimate with the weight value of  $\tau$ , penalizes underestimate with the weight of  $1-\tau$ .

14:   Update the rule  $\theta(x, i) = \theta(x, i) + \alpha(\tau_i|\{\sum_{t=0}^{t-\infty} \gamma^t R_t \geq \theta(x, i)\} - (1-\tau_i)|\{\sum_{t=0}^{t-\infty} \gamma^t R_t \leq \theta(x, i)\})$ , where  $\theta(x, i)$  is the  $i$ th predicted quantile value of state  $x$ ,  $\alpha$  is the learning rate.

15:   Output the updated rule  $\theta'(x, i) = \theta(x, i) + \alpha \frac{1}{m} \sum_{j=1}^m (\tau_i - 1\{r + \gamma\theta(x', j) - \theta(x, i) < 0\}) + H(s, a)$

16: **End for**

17:   Set  $\theta(x, i) \leftarrow \theta'(x, i)$

18: **End for**

19:   Generate the updated trained quantile value  $\theta'(x, i)_{i=1}^m, x \in X$

20: **End Training phase of SLM\_QTDL**

21: **Testing phase of SLM\_QTDL**

22: **For** each of the applications in the application set, i.e.,  $IoT_{App_i} \in IoT_{App}$  **do**

23:   Determine quantile regression loss of line number 11

24:   Compute quantile regression loss of line number 12

25:   Unbiased estimator of negative gradient loss of line number 13

26:   Update the rule of line number 14

27:   Output the updated rule  $\theta'(x, i) = \theta(x, i) + \alpha \frac{1}{m} \sum_{j=1}^m (\tau_i - 1\{r + \gamma\theta(x', j) - \theta(x, i) < 0\}) + H(s, a)$

28: **End for**

29:   Set  $\theta(x, i) \leftarrow \theta'(x, i)$

30: **End for**

31:   Generate the updated tested quantile value  $\theta'(x, i)_{i=1}^m, x \in X$

32: **End Testing phase of SLM\_QTDL**

33: **Output**  $IoT\_Pol = \{IoT_{App_1} \rightarrow fn_1, IoT_{App_2} \rightarrow fn_2, IoT_{App_3} \rightarrow fn_3, \dots, IoT_{App_m} \rightarrow fn_n\}$

34: **Stop**

**5. Expected Value Analysis**

The expected value analysis of the proposed SLM\_QTDL framework is carried out considering a typical fog computing scenario. It is common practice in decision theory to determine the outcomes in an uncertain scenario using probability-weighted values.

These are the average values that are obtained after repeated conduct of the experiment. The performance of the SLM\_QTDL is compared with that of three recent frameworks: Memetic Algorithm (MA) [12], Prioritization Model (PM) [6], and Distributed Deep Learning (DDL) [15]. The performance is evaluated considering the performance metrics of total energy consumption, makespan time violation, budget violation, and load imbalance.

PM1: Energy consumption ( $EC(SLM\_QTDL)$ ): The energy consumption of the SLM\_QTDL is influenced by the value of  $SEC(SLM\_QTDL, \Delta T)$  and  $DEC(SLM\_QTDL, \Delta T)$ .

$$\begin{aligned}
 EV\left(\frac{EC(SLM\_QTDL)}{\pi}, \Delta T\right) &= \int_x^y \frac{\sum_{a \in \pi} EC(SLM\_QTDL)(a)}{|\pi|} \\
 EV\left(\frac{EC(SLM\_QTDL)}{\pi}, \Delta T\right) &= \frac{EV(1 * \pi * SEC(SLM\_QTDL, \Delta T) + DEC(SLM\_QTDL, \Delta T))}{\pi} \\
 &= \sum_{d \in D} d \int_q^Q \left[ (\alpha, 0) + \sum_{fn_i \in FN} fn_i^{CPU}(\Delta T) * MAX(EC(fn_i)) \right] \\
 &= \int_q^Q Q * dP \frac{[(\alpha, 0) + \sum_{fn_i \in FN} fn_i^{CPU}(\Delta T) * MAX(EC(fn_i))]}{\pi} \\
 EC(SLM\_QTDL): EV\left(\frac{EC(SLM\_TDQL)}{\Pi}, T\right) &\cong low \\
 EC(MA): EV\left(\frac{EC(MA)}{\Pi}, T\right) &\cong High \\
 EC(PM): EV\left(\frac{EC(PM)}{\Pi}, T\right) &\cong Medium \\
 EC(DDL): EV\left(\frac{EC(DDL)}{\Pi}, T\right) &\cong High
 \end{aligned}$$

The  $EC(SLM\_QTDL)(a)$  represents the energy consumption of the model after taking the agent action  $a$ . The energy is averaged over the policy  $\pi$  and integrated with respect to time. The energy consumption modeling is averaged across all IoT devices. By integrating the quantiles, the distributional returns are formulated to incorporate risk awareness inside the TD learning model. The model captures both the mean behavior and risk-causing behavior of the policy formulated. Both the quantile space and probability destiny of quantile returns capture accurate estimation of the energy consumed under dynamically varying fog node characteristics.

PM2: Makespan time violation ( $MSV(SLM\_TDQL)$ ): The makespan time violation of the SLM\_QTDL is influenced by the value of maximum value of makespan time  $MAX(MS\_SLM\_QTDL)$  and makespan time as defined by the QoS profile  $MS\_QoS(App_i)$ .

$$\begin{aligned}
 EV\left(\frac{MSV(SLM\_TDQL)}{\pi}, \Delta T\right) &= \int_x^y \frac{\sum_{a \in \pi} MAX(MS\_SLM\_QTDL)(a) - MS\_QoS(App_i)(a)}{|\pi|} \\
 EV\left(\frac{MSV(SLM\_TDQL)}{\pi}, \Delta T\right) &= \sum_{d \in D} d \int_x^y \frac{\sum_{a \in \pi} MAX(MS\_SLM\_QTDL)(a) - MS\_QoS(App_i)(a)}{|\pi|} \\
 EV\left(\frac{MSV(SLM\_TDQL)}{\pi}, \Delta T\right) &= \frac{E(1 * \pi * MAX(MS\_SLM\_QTDL)(a) - MS\_QoS(App_i)(a))}{P(\pi)} \\
 EV\left(\frac{MSV(LLM\_QL)}{\pi}, \Delta T\right) &= \sum_{d \in D} d \int_q^Q [MAX(MS\_SLM\_QTDL)(a) - MS\_QoS(App_i)(a)] \\
 EV\left(\frac{MSV(LLM\_QL)}{\pi}, \Delta T\right) &= \int_q^Q Q * dP \frac{[MAX(MS\_SLM\_QTDL)(a) - MS\_QoS(App_i)(a)]}{\pi}
 \end{aligned}$$

$$MSV(SLM\_TDQL): EV\left(\frac{MSV(SLM\_TDQL)}{\Pi}, T\right) \cong Low$$

$$MSV(MA) = EV\left(\frac{MSV(MA)}{\Pi}, T\right) \cong High$$

$$MSV(PM) = EV\left(\frac{MSV(PM)}{\Pi}, T\right) \cong High$$

$$MSV(DDL) = EV\left(\frac{MSV(DDL)}{\Pi}, T\right) \cong Medium$$

The  $MSV(SLM\_QTDL)(a)$  represents the expected violation of the makespan time after taking action  $a$ . The average of makespan time violation is computed considering both worst case execution time and targeted quality of service (QoS) which is aggregated over the policies formulated. The makespan time violation parameters are combined with the distribution of return quantiles to represent the uncertainty of the fog environment. The model is made risk aware by capturing the uncertainty distributional form of temporal difference learning. The effective measurement of makespan time is ensured by deviating the learned application placement policies according to the time sensitive QoS parameters.

PM3: Budget Violation ( $BV(SLM\_QTDL)$ ): The budget violation of the  $SLM\_QTDL$  is influenced by cost of executing the application on fog node  $Cost_{App_i}^{fn_i}$  and pricing model followed at the fog node  $Price_{fn_i}$ .

$$EV\left(\frac{BV(SLM\_QTDL)}{\pi}, \Delta T\right) = \int_x^y \frac{\sum_{a \in \pi} BV(SLM\_QTDL)(a)}{|\pi|}$$

$$EV\left(\frac{BV(SLM\_QTDL)}{\pi}, \Delta T\right) = \sum_{d \in D} d \int_x^y \frac{\sum_{a \in \pi} BV(SLM\_QTDL)(a)}{|\pi|}$$

$$EV\left(\frac{BV(SLM\_QTDL)}{\pi}, \Delta T\right) = \frac{EV\left(1 * \pi * \sum_{i=1}^{i=m} Cost_{App_i}^{fn_i} * Price_{fn_i}\right)}{P(\pi)}$$

$$EV\left(\frac{BV(SLM\_QTDL)}{\pi}, \Delta T\right) = \sum_{d \in D} d \int_q^Q \sum_{i=1}^{i=m} Cost_{App_i}^{fn_i} * Price_{fn_i}$$

$$EV\left(\frac{BV(SLM\_QTDL)}{\pi}, \Delta T\right) = \int_q^Q Q * dP \frac{\sum_{i=1}^{i=m} Cost_{App_i}^{fn_i} * Price_{fn_i}}{\pi}$$

$$BV(SLM\_QTDL): EV\left(\frac{BV(SLM\_QTDL)}{\Pi}, T\right) \cong Low$$

$$BV(MA) = EV\left(\frac{BV(MA)}{\Pi}, T\right) \cong Medium$$

$$BV(PM) = EV\left(\frac{BV(PM)}{\Pi}, T\right) \cong Low$$

$$BV(DDL) = EV\left(\frac{BV(DDL)}{\Pi}, T\right) \cong High$$

The  $BV(SLM\_QTDL)(a)$  represents the budget violation of  $SLM\_QTDL$  agent after executing the action  $a$ . The average over the budget violation is performed with respect to time and over all policy actions taken. The edge resources cost and each node pricing is averaged across all applications. The distributional quantile aware resource pricing makes the policy adaptable to risk sensitive situations. The variability in the price and cost are captured by integrating quantile space along with the probability density.

PM4: Load imbalance ( $LI(SLM\_QTDL)$ ): The load imbalance of the  $SLM\_QTDL$  is influenced by CPU utilization  $fn_i^{CPU}(\Delta T)$ , memory utilization  $fn_i^{MU}(\Delta T)$ , and bandwidth utilization  $fn_i^{BU}(\Delta T)$ .

$$EV\left(\frac{LI(SLM\_QTDL)}{\pi}, \Delta T\right) = \int_x^y \frac{\sum_{a \in \pi} LI(SLM\_QTDL)(a)}{|\pi|}$$



$$\begin{aligned}
EV\left(\frac{LI(SLM\_QTDL)}{\pi}, \Delta T\right) &= \sum_{d \in D} d \int_x^y \frac{\sum_{a \in \pi} LI(SLM\_QTDL)(a)}{|\pi|} \\
E\left(\frac{LI(SLM\_QTDL)}{\pi}, \Delta T\right) &= \frac{E\left(1 * \pi * \frac{fn_i^{CPU}(\Delta T)}{fn_i^{CPU}(\Delta T) + fn_i^{MU}(\Delta T) + fn_i^{BU}(\Delta T)}\right)}{P(\pi)} \\
E\left(\frac{LI(SLM\_QTDL)}{\pi}, \Delta T\right) &= \frac{E\left(1 * \pi * \frac{fn_i^{MU}(\Delta T)}{fn_i^{CPU}(\Delta T) + fn_i^{MU}(\Delta T) + fn_i^{BU}(\Delta T)}\right)}{P(\pi)} \\
E\left(\frac{LI(SLM\_QTDL)}{\pi}, \Delta T\right) &= \frac{E\left(1 * \pi * \frac{fn_i^{BU}(\Delta T)}{fn_i^{CPU}(\Delta T) + fn_i^{MU}(\Delta T) + fn_i^{BU}(\Delta T)}\right)}{P(\pi)} \\
&= \sum_{d \in D} d \int_q^Q \alpha_1 * fn_i^{CPU}(\Delta T) + \alpha_2 * fn_i^{MU}(\Delta T) + \alpha_3 * fn_i^{BU}(\Delta T) \\
&= \int_q^Q Q * dP \frac{[\alpha_1 * fn_i^{CPU}(\Delta T) + \alpha_2 * fn_i^{MU}(\Delta T) + \alpha_3 * fn_i^{BU}(\Delta T)]}{\pi}
\end{aligned}$$

$$LI(SLM\_QTDL): EV\left(\frac{LI(SLM\_QTDL)}{\pi}, T\right) \cong Low$$

$$LI(MA) = EV\left(\frac{LI(MA)}{\pi}, T\right) \cong High$$

$$LI(PM) = EV\left(\frac{LI(PM)}{\pi}, T\right) \cong Medium$$

$$LI(DDL) = EV\left(\frac{LI(DDL)}{\pi}, T\right) \cong Medium$$

The LI(SLM\_QTDL) captures the contribution of each of the resources towards load imbalance. The CPU and memory bandwidths of each of the fog nodes are measured over the time interval. The weighted coefficients  $\alpha_1$ ,  $\alpha_2$ ,  $\alpha_3$  represent the operational priority and by aggregating them, a unified value for the load imbalance metric is obtained. By assigning distributional weights over the resource components a risk-sensitive load-balancing model is developed. The stochastic variance in the resource usage is captured through the combination of quantile returns and probability densities. The overall system resilience is assured by minimizing the chances of load imbalance through quantile aware application placement policies.

## 6. Results and Discussion

The proposed SLM\_QTDL framework is experimentally evaluated using the iFogSim simulator [18–20]. The simulation duration is 120 s to 240 s. The simulation parameters are initialized as shown in Table 1. The QTDL model is trained externally using the Python 3.13.7 scikit learn package. The iFogSim java code is modified to read the QTDL model output. The output generated is used to train the QTDL model which predicts the resource requirement of the incoming applications. The Java machine learning libraries are embedded which include Weka and Deeplearning4j. The custom modules embedded inside the machine learning logic are ModulePlacement and APPModuleScheduler. The QTDL decision steps are selected by modifying controller.java. The iFogSim and QTDL model are run in parallel. For inter process communication, REST API and sockets are used.

**Table 1.** Simulation parameter details.

| Simulation Parameters                                    | Initialized Value                                                                    |
|----------------------------------------------------------|--------------------------------------------------------------------------------------|
| Minimum number of applications                           | 100                                                                                  |
| Maximum number of applications                           | 9000                                                                                 |
| Bandwidth of WLAN                                        | 500 Mbits/sec                                                                        |
| End device counter step                                  | 100                                                                                  |
| Task generation rate                                     | 5 task/device/minutes                                                                |
| Number of fog nodes                                      | 10,000                                                                               |
| Propagation delay                                        | 0.2 s                                                                                |
| Bandwidth of WAN                                         | 500 Mbits/sec                                                                        |
| Request length                                           | 50–300 MI                                                                            |
| Request size                                             | 1–16 megabits                                                                        |
| Processing power of IoT devices                          | 1000–2000 MIPS                                                                       |
| RAM memory size of IoT devices                           | 256–312 MB                                                                           |
| processing power of IoT devices                          | 1000–2000 MIPS                                                                       |
| RAM memory size of IoT devices                           | 256–512 MB                                                                           |
| Processing power of fog nodes                            | 4000–8000 MIPS                                                                       |
| RAM memory size of fog nodes                             | 512–1024 MB                                                                          |
| Processing power of each of the cloud server             | 12,000–30,000 MIPS                                                                   |
| RAM memory size of cloud server                          | 2048–4096 MB                                                                         |
| Status sharing and observation period                    | 10 s                                                                                 |
| Application to fog nodes                                 | 10–15 ms                                                                             |
| Fog nodes to cloud nodes                                 | 140–160 ms                                                                           |
| The application service delivery deadline                | 350 ms to 750 ms                                                                     |
| Maximum nodal communication delay within the fog network | 3–5 ms                                                                               |
| Application data receiving frequency                     | 3 s to 7 s                                                                           |
| Total energy consumed                                    | 680 Joules (summation of energy consumed by IoT devices + Fog nodes + Cloud devices) |
| Average energy consumed by the application               | 15 Joules/ Application                                                               |
| Energy consumed by fog node                              | 200 Joules                                                                           |
| Energy consumed by IoT device                            | 280 Joules (sensing of data and transmission of data)                                |
| Transmission energy                                      | 180 Joules (transmission of data from IoT devices to fog nodes then to cloud)        |
| Average time taken for application execution             | 1.5 s                                                                                |
| Average latency incurred for each application = 1.0 s    | 1.0 s                                                                                |
| Application deadline miss rate                           | 1%                                                                                   |
| Fog CPU utilization                                      | 80%                                                                                  |

Table 1. Cont.

| Simulation Parameters                   | Initialized Value                                         |
|-----------------------------------------|-----------------------------------------------------------|
| IoT device failure rate                 | 5%                                                        |
| Average recovery time                   | 3.5 s                                                     |
| Average application task execution time | 1.5 s                                                     |
| Network throughput                      | 2.5 Mbps                                                  |
| Rescheduling of tasks due to failures   | 5 Tasks                                                   |
| Energy-delayed product                  | 18.2 Joules Seconds/task                                  |
| Fog node utilization                    | 85%                                                       |
| Compute resources                       | CPU = 45%, Memory = 69%, threads = 115                    |
| Network                                 | Latency = 50 ms, Bandwidth = 1.2 Mbps, packet loss = 0.2% |
| Storage                                 | Disk I/O = 10 MB/s                                        |
| IoT device health                       | Battery = 38%, temperature = 56 °C, uptime = 3 h          |
| System events                           | Task failed = 1, overload alert = true                    |

#### PM1: Energy consumption

A graph of the number of applications versus energy consumption is shown in Figure 2. It is observed from the graph that the energy consumption of the SLM\_QTDL is consistently less with the increase in the number of applications. The SLM-enriched QTDL needs lesser processing power and is trained very well even with the availability of limited computational resources. Also, it prevents the sharp increases in the TD error through faster and stable reward convergence. Meanwhile, the confidence interval is 95%, i.e., 95 samples among 100 samples contain true average. The 95% confidence interval for SLM\_QTDL is [2000–6000] Joules. The energy consumption of MA kept increasing with the increase in the number of applications. As it becomes stuck in local optima due to the use of local search component for decision making, the 95% percentage confidence interval of MA is [3500–13,000] Joules. The energy consumption of PM and DDL remain moderate with the increase in the number of applications. The frequent synchronization between gradient parameters slows down the training process and leads to convergence to suboptimal solutions. The 95% confidence intervals of PM and DDL are [5000–14,000].

#### PM2: Makespan time violation

A graph of the number of fog nodes versus makespan time violations is given in Figure 3. It is observed from the graph that the makespan time violation frequency is less for SLM\_TDL with the increase in the number of fog nodes, because they incur a fast processing time and converge to an optimal estimation of Q function value. The makespan time violation frequency is moderate for MA and PM. Due to the use of a higher intensity, the local search mechanism and finer tuning of parameters incur a longer time for processing; meanwhile, the makespan time violation frequency is very high for DDL with the increase in the number of fog nodes. Its communication overhead is greater and often ends up in a performance bottleneck due to unstable training.

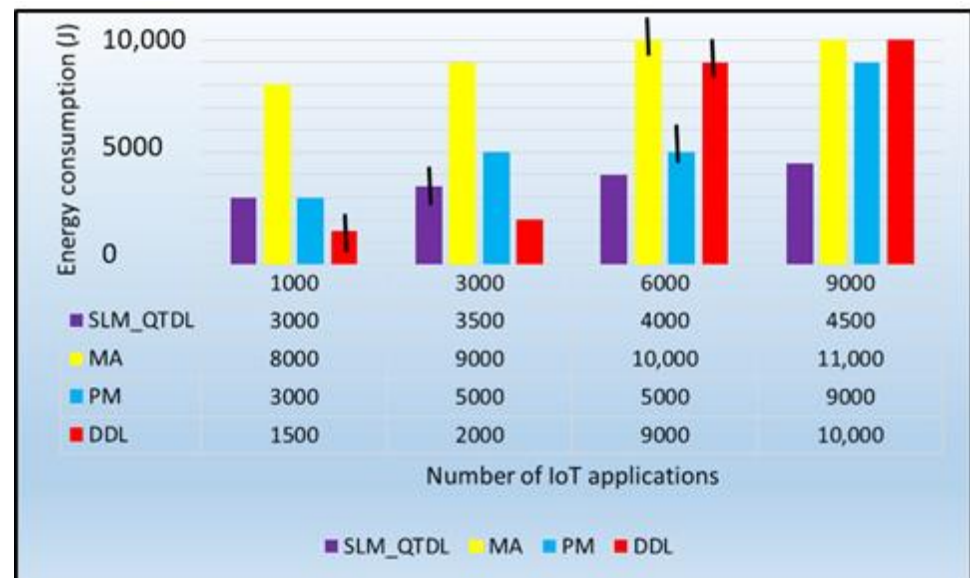


Figure 2. Number of applications versus energy consumption (Joules).

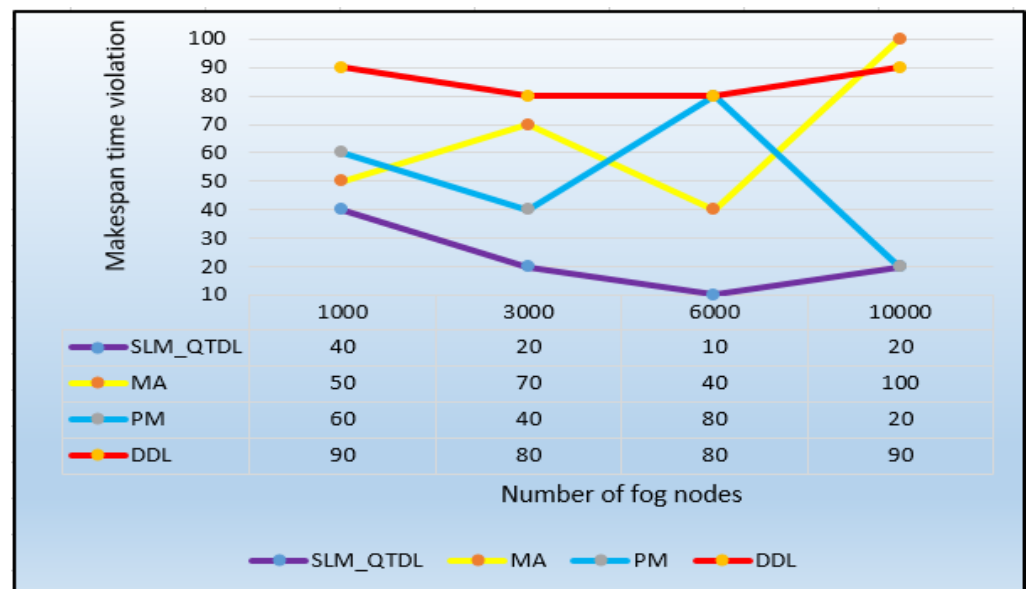
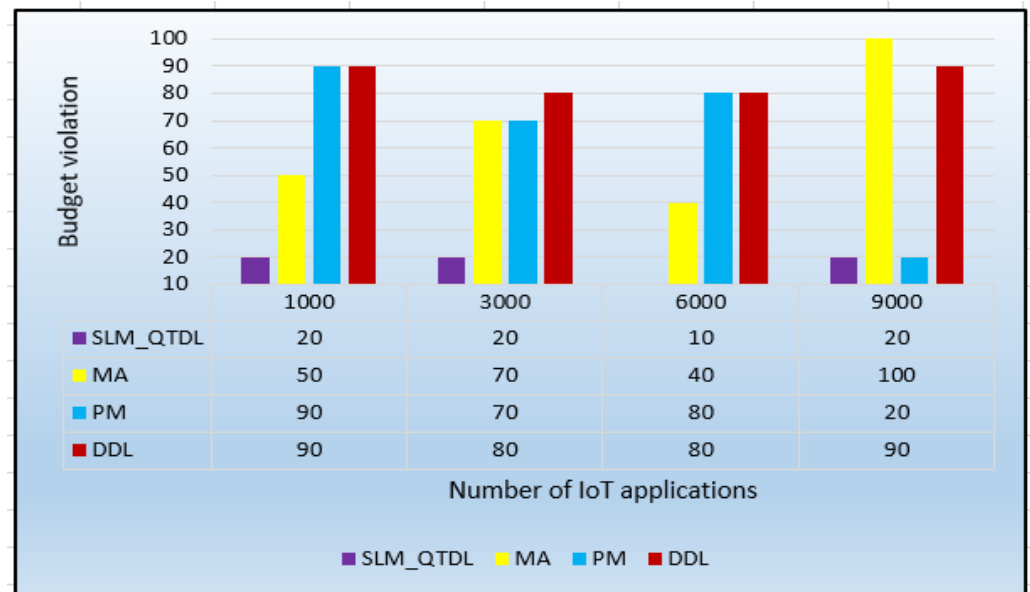


Figure 3. Number of fog nodes versus makespan time (ms).

### PM3: Budget violation

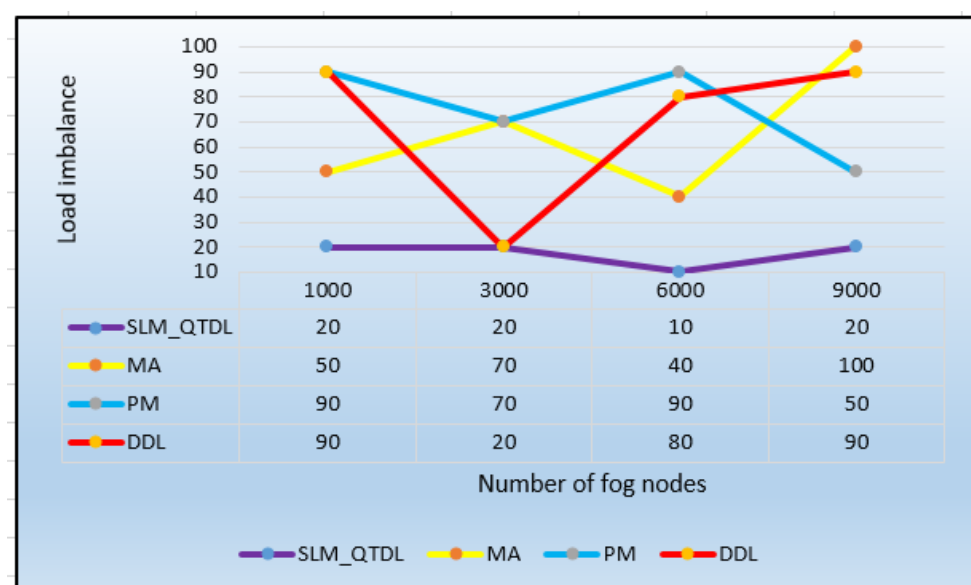
A graph of the number of applications versus budget violations is given in Figure 4. It is inferred from the graph that the budget violation frequency is kept consistently lesser for SLM\_TDL with the increase in the number of applications as it uses quantile regression and TD learning that are validated using the heuristics. The budget violation frequency remains moderate for MA. The local search mechanisms are highly expensive and eventually slow down the working of memetic logic. The budget violation frequency remains very high for both PM and DDL and due to higher operation capacity and flexibility, they are often prone to overfitting problems.



**Figure 4.** Number of applications versus budget violations.

#### PM4: Load imbalance

A graph of the number of fog nodes versus load imbalance is shown in Figure 5. It is inferred from the graph that the load imbalance frequency is much lower for SLM\_QTDL because it employs distributed reinforcement learning policies and evaluates the policies using a discounted summation of the rewards for a better exploration of the state space. Also the temporal difference error is less due to proper controlling of hallucination by propagating only true beliefs. The load imbalance frequency of MA remains moderate because it does not scale automatically for a higher dimensional problem; meanwhile, the load imbalance frequency of PM and DDL keeps increasing with the increase in the number of fog nodes. Due to the incorporation of gradient staleness during training, it often leads to higher instability in training and often the model behaves in an erratic operation.



**Figure 5.** Number of fog nodes versus load imbalance (0–1).

Through implementation, it is observed that the powerful combination of a small language model and temporal difference learning successfully prevents overestimation or underestimation of the Q values. Through proper reward shaping, an ambiguous and noisy state space environment is traversed to arrive at the proper decisions. The applications placement policies are found to be accurate and, in turn, improve the overall system performance with respect to energy consumption, makespan time violations, budget violations, and load imbalance ratio.

## 7. Conclusions

This paper presents a novel small language model-guided temporal difference learning framework designed to perform optimal application placement in fog computing. The heuristic value generated by the SLM is used to learn the exact Q value of TD learning. This prevents the tendency to enter into hallucination mode and biased decisions by avoiding the improper guidance to TD learning agent. Also the stability is ensured in Q value variance, an improved learning rate, and a decline in the temporal difference error. The SLM-enriched Q value offers a greater level of flexibility through proper reshaping of reward function. The experimental results obtained by the iFogSim 3.3. simulator are good with respect to energy consumption, makespan time violations, budget violations, and load imbalance. However, the SLMs are trained over significantly less data which lacks better understanding of the ongoing dynamics of the fog environment. Small models consist of fewer parameters which limit their applicability and cannot handle ambiguous forms of the applications. As the models are not fine-tuned, they need to be explicitly adapted to the resource-constrained fog environment. In future work, we aim to enhance the scalability of the framework to handle heterogeneous resources and efficiently distribute applications among the fog nodes by satisfying the QoS and QoE constraints of the end users. By fine-tuning the small language models, the in-context and out-context learning will be performed effectively to act under uncertainty in the operating environment. To handle the monitoring issues, lightweight monitoring agents will be incorporated to design a unified framework. A score will be attached to every fog node based on uptime history, task success rate, and failure frequency to choose reliable fog nodes for application placement.

**Author Contributions:** Conceptualization, B.K. and S.G.S.; methodology, B.K.; software, S.G.S.; validation, S.G.S.; formal analysis, B.K.; investigation, B.K.; resources, S.G.S.; data curation, B.K.; writing—original draft preparation, B.K.; writing—review and editing, S.G.S.; visualization, B.K.; supervision, S.G.S.; project administration, S.G.S. All authors have read and agreed to the published version of the manuscript.

**Funding:** This research received no external funding.

**Data Availability Statement:** The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

**Conflicts of Interest:** The authors declare no conflict of interest.

## References

1. Srirama, S.N. A decade of research in fog computing: Relevance, challenges, and future directions. *Softw. Pract. Exp.* **2024**, *54*, 3–23. [\[CrossRef\]](#)
2. Pozzebon, A. Edge and fog computing for the Internet of Things. *Future Internet* **2024**, *16*, 101. [\[CrossRef\]](#)
3. Khan, S.; Shah, I.A.; Ahmad, S.; Khan, J.A.; Anwar, M.S.; Aurangzeb, K. A Comprehensive Survey on Multi-Facet Fog-Computing Resource Management Techniques, Trends, Applications and Future Directions. *Expert Syst.* **2025**, *42*, e70019. [\[CrossRef\]](#)
4. Refonaa, J.; Maheswari, M.; Poornima, D.; Shabu, S.J.; Gowri, M.; Praveen, S.; Amshavalli, R.S. Edge and Fog Computing for Sustainability. In *Energy Efficient Algorithms and Green Data Centers for Sustainable Computing*; IGI Global Scientific Publishing: Hershey, PA, USA, 2025; pp. 79–108.



5. Apat, H.K.; Goswami, V.; Sahoo, B.; Barik, R.K.; Saikia, M.J. Fog Service Placement Optimization: A Survey of State-of-the-Art Strategies and Techniques. *Computers* **2025**, *14*, 99. [\[CrossRef\]](#)
6. Mann, Z.A. Decentralized application placement in fog computing. *IEEE Trans. Parallel Distrib. Syst.* **2022**, *33*, 3262–3273. [\[CrossRef\]](#)
7. Goswami, A.; Modi, K.; Patel, C. Evaluation of Optimization Algorithm for Application Placement Problem in Fog Computing: A Systematic Review. *Arch. Comput. Methods Eng.* **2025**, *32*, 1–29. [\[CrossRef\]](#)
8. Rowland, M.; Munos, R.; Azar, M.G.; Tang, Y.; Ostrovski, G.; Harutyunyan, A.; Dabney, W. An analysis of quantile temporal-difference learning. *J. Mach. Learn. Res.* **2024**, *25*, 1–47.
9. Peng, Y.; Zhang, L.; Zhang, Z. Statistical efficiency of distributional temporal difference learning. *Adv. Neural Inf. Process. Syst.* **2024**, *37*, 24724–24761.
10. Javaheripi, M.; Bubeck, S.; Abidin, M.; Aneja, J.; Bubeck, S.; Mendes, C.C.T.; Gopi, S. Phi-2: The surprising power of small language models. *Microsoft Res. Blog* **2023**, *1*, 3.
11. Hu, S.; Tu, Y.; Han, X.; He, C.; Cui, G.; Long, X.; Sun, M. Minicpm: Unveiling the potential of small language models with scalable training strategies. *arXiv* **2024**, arXiv:2404.06395. [\[CrossRef\]](#)
12. Goudarzi, M.; Wu, H.; Palaniswami, M.; Buyya, R. An application placement technique for concurrent IoT applications in edge and fog computing environments. *IEEE Trans. Mob. Comput.* **2020**, *20*, 1298–1311. [\[CrossRef\]](#)
13. Nashaat, H.; Ahmed, E.; Rizk, R. IoT application placement algorithm based on multi-dimensional QoE prioritization model in fog computing environment. *IEEE Access* **2020**, *8*, 111253–111264. [\[CrossRef\]](#)
14. Eyckerman, R.; Reiter, P.; Latré, S.; Marquez-Barja, J.; Hellinckx, P. Application placement in fog environments using multi-objective reinforcement learning with maximum reward formulation. In *Proceedings of the NOMS 2022–2022 IEEE/IFIP Network Operations and Management Symposium, Budapest, Hungary, 25–29 April 2022*; IEEE: New York, NY, USA, 2022; pp. 1–6.
15. Goudarzi, M.; Palaniswami, M.; Buyya, R. A distributed deep reinforcement learning technique for application placement in edge and fog computing environments. *IEEE Trans. Mob. Comput.* **2021**, *22*, 2491–2505. [\[CrossRef\]](#)
16. Gu, X.; Wu, Q.; Fan, P.; Cheng, N.; Chen, W.; Letaief, K.B. Drl-based federated self-supervised learning for task offloading and resource allocation in isac-enabled vehicle edge computing. *Digit. Commun. Networks* **2024**. [\[CrossRef\]](#)
17. Sacoto-Cabrera, E.J. Fog Computing: A Systematic Review from the Perspectives of Architecture, Business Models, and Research Developments. In *Proceedings of the 2024 IEEE Colombian Conference on Communications and Computing (COLCOM), Barranquilla, Colombia, 21–23 August 2024*; IEEE: New York, NY, USA, 2024; pp. 1–6.
18. Gupta, H.; Vahid Dastjerdi, A.; Ghosh, S.K.; Buyya, R. iFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, Edge, and Fog computing environments. *Softw. Pract. Exp.* **2017**, *47*, 1275–1296. [\[CrossRef\]](#)
19. Elsayed, A.; Mohamed, K.; Harb, H. Enhanced Traffic Congestion Management with Fog Computing: A Simulation-based Investigation using iFog-Simulator. *arXiv* **2023**, arXiv:2311.01181. [\[CrossRef\]](#)
20. Awaisi, K.S.; Abbas, A.; Khan, S.U.; Mahmud, R.; Buyya, R. Simulating fog computing applications using iFogSim toolkit. In *Mobile Edge Computing*; Springer: Berlin/Heidelberg, Germany, 2021; pp. 565–590.

**Disclaimer/Publisher’s Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.